WebSailor深度解析——让开源模型具备超人类网页推理能力

📑 论文 | 💾 代码

背景

过去两年，大模型的能力快速提升，虽然各大公司还在不断推出新的模型，但也逐渐遇到了“天花板”：

- 📉 **数据枯竭**：互联网上的文本几乎被用尽，只能依赖合成数据继续增强。
- ❓ **任务更复杂**：人们希望模型能处理跨领域、模糊、实时的问题，但仅靠模型“记忆”已不足以应对。

于是，人们开始探索：如何让大模型像人类研究员一样，学会主动利用外部信息，进行检索、分析和推理？这就引出了 DeepResearch 这一方向。

大模型的“检索进化史”

如果把大模型的发展按“检索与推理能力”来划分，大致经历了以下几个阶段：

1. 🧮 **基础大模型（无检索能力）**

   直接依靠大模型自身回答问题，但受限于训练数据的时间范围，无法回答最新事件。

2. 🔎 **检索增强（RAG/Web 搜索）**

   将用户问题转成搜索词，检索互联网或知识库的数据，将其结果作为大模型输入。

   方式较为简单，基本上是单步搜索（当然做一些prompt工程也可以多步搜索），可回答时效性强或私域知识的问题。

   📌 举个例子：

   - 问题：2025 年世界人工智能大会（WAIC）的主题是什么？
   - 系统：直接搜索 → 找到新闻 → 模型总结 → 回答“智能时代 同球共济”。

3. 🧠 **推理驱动的检索**

   模型不再一股脑儿搜索，而是先拆解问题，把问题分解成多步查询，逐步拼接答案，能应对稍复杂的任务，但依旧通过 prompt 工程控制。

   📌 举个例子：

- 提问：“现任特斯拉 CEO 的母校所在城市的标志性建筑是什么？”
- 系统：
  i. 搜“现任特斯拉 CEO” → 得到“埃隆·马斯克”；
  ii. 搜“埃隆·马斯克 本科 母校” → 宾夕法尼亚大学；
  iii. 搜“宾夕法尼亚大学所在城市 标志性建筑” → 费城 + 独立厅；
  iv. 最终回答：费城独立厅

4. 🤖 **内嵌检索 Agent**
   检索不再靠 prompt 手工提示，而是训练模型学会「何时该检索、检索什么」。 实际效果与第三种类似，没有区分任务难度专门训练。

5. 🔬 **DeepResearch（研究式检索推理）**
   专门为高度模糊、跨领域、多跳问题设计，通过强化学习进一步增强模型推理、反思、总结等能力。需要进行多步循环式研究流程：**思考 → 检索 → 整理 → 反思 → 再检索 → 逐步缩小范围 → 得出结论。**

   它可以像人类研究员一样，不断假设、验证、修正，甚至相比我们普通人检索总结的能力更强。要注意，这种方法是专门为复杂问题设计的，所以没必要用 DeepResearch 问一个很简单的问题。

   ✨ 注意：很多文章说 DeepResearch 会输出一个研究报告，我认为这只是一个展现形式，让人们看到多步推理总结的过程。它的核心还是解决复杂问题，只输出几个字的最终结果也算 DeepResearch。

   📌 举个例子：
   - 提问：“一首与欧洲首都相关的歌曲，其曲作者曾在 2010 年代获得该城市文化奖，并且就读于英国北部的音乐学院，这首歌叫什么？”
   - DeepResearch 系统的流程：
     i. 初步假设：欧洲首都 → 伦敦、巴黎、柏林…
     ii. 检索“伦敦 2010年代 文化奖 词曲作者” → 找到候选人。
     iii. 过滤条件：“是否就读于英国北部音乐学院” → 缩小到皇家北方音乐学院校友。
     iv. 再检索他们的代表作品 → 《雾都回响》。
   - ✅ 最终回答：《雾都回响》。

BrowseComp 基准测试

- OpenAI 在推出 **DeepResearch** 的同时，还推出了超难的基准测试——**BrowseComp**。这是一个网页推理任务的基准测试集，主要用于评估模型在复杂网页任务中的表现。
- 此前，阿里也发布过 WebDancer 模型，但还不敢“叫板”BrowseComp，因为和大家一样分数都太低了（个位数）。

- 直到 WebSailor 出现，才第一次让开源模型在 BrowseComp 榜单上站稳了一席之地，（也仅仅是一席，因为相较于 OpenAI 的 DeepResearch 还是差的很多），这是开源世界在 **复杂网页推理** 上的一次重要突破。

问题定义

数据难度分析

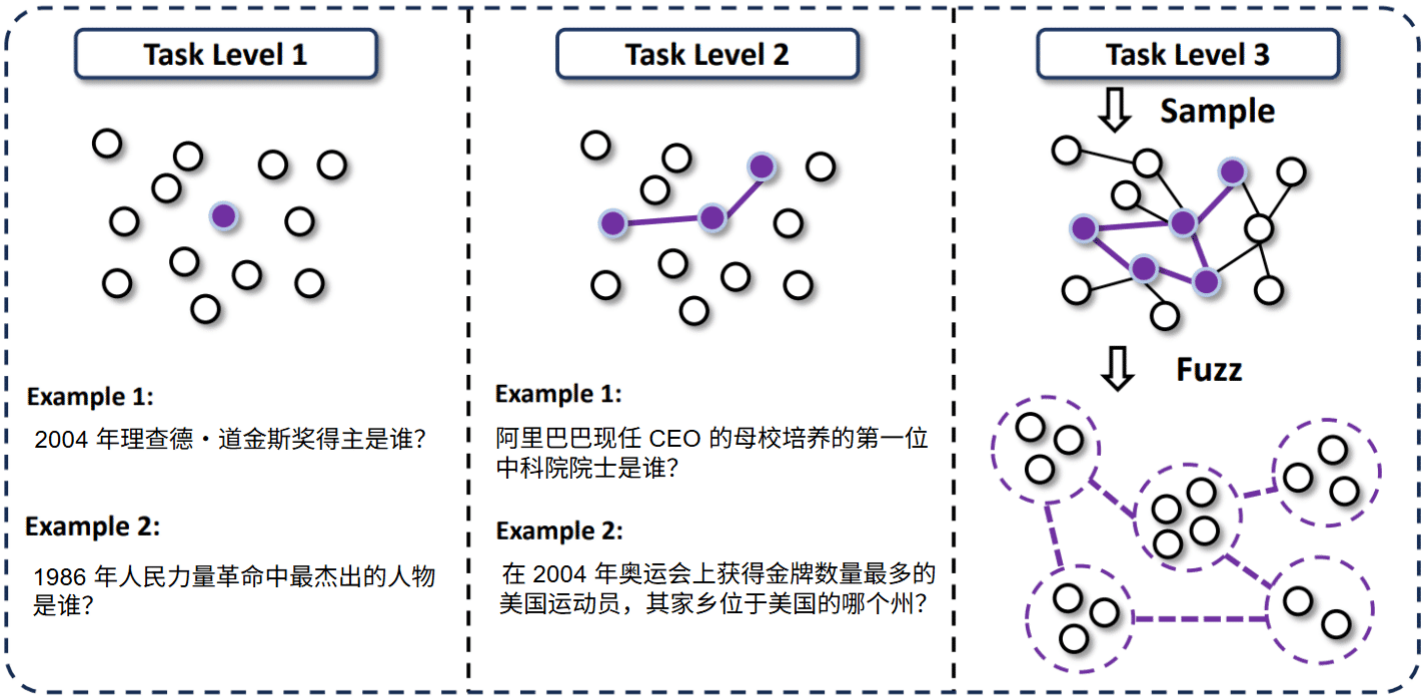
为什么BrowseComp这么难，WebSailor 进行了分析

一个问题能不能搜出来答案，两个因素：

1. 这个问题本身的不确定性（可能就是没有答案）
2. 降低问题不确定性的复杂程度（一个复杂隐晦的问题搜几次就可以变的更明确）。

他将现在的信息搜索类问题按照这两个维度分为3个等级：

等级1 ✅	等级2🔄	等级3🕸
• **不确定性低，容易降低。** • 模型内部知识 or 一次简单搜索即可解决。 • 示例：新闻类、事实类直接问题。	• **初始不确定性高，但可以逐步降低。** • 问题逻辑呈 线性链，一步步搜索即可得到答案。 • 示例：多步事实链推理。	• 不确定性高，降低难度也高。 • 问题实体之间关联复杂、隐晦，呈 **非线性拓扑**。 • 模型必须探索式检索 + 创造性推理。



问题难度等级对比

现有模型的困境

- 当前开源模型训练时，主要数据分布集中在 **等级 1 和 2**。
- **等级 3 几乎未见过 → 导致模型在 BrowseComp 上得分普遍只有个位数。**
- 这正是 WebSailor 要突破的核心难点。

准备数据集

SailorFog-QA：用知识图谱合成 QA 对

WebSailor 的核心突破点在于构造了一个极具挑战性的 **SailorFog-QA 数据集**，它专门针对 **等级 3 问题**（高不确定性 + 高复杂性）而设计。

按照上图中表示，问题的难度可以用知识图谱的形式表示，Task Level 3的问题会表现成非线性拓扑的结构。**那就从知识图谱出发，先在图中选择一个起点，然后发散性的选择周围的一些点，让大模型去扩散链路生成QA。**

WebSailor 使用的是 **维基百科的 Wikidata**，它是一个大型开源知识图谱，可以通过 **SPARQL 查询语句**进行操作。

构造步骤 🔨

1️⃣ **选择模糊实体作为起点** 🎯

- 判断方法：如果某个实体有多个类型，就可以视为模糊实体。
- 示例：`苹果` → 既是水果 🍎，也是手机品牌 📱。
- 示例：`埃菲尔铁塔` → 类型单一（建筑），确定性强，不适合作为起点。
- 可以通过 SPARQL 的规则来筛选出这类模糊或稀有实体，保证起点带有天然不确定性。

2️⃣ **模拟网页浏览，构建初始知识图谱** 🌍

- 用第一步得到的词进行网页搜索。
- 提取与它相关的实体和关系，形成新的节点和边。

3️⃣ **迭代扩展** 🔄

- 随机选择现有节点，继续扩展出不同的实体。
- 刻意避免“单一路径”（如：苹果→iPhone→iOS，这种更像 Level 2 问题）。

- 转而构造“密集交叉网络”：例如：
  - `苹果` → 水果 / 电子品牌 / 乔布斯 / 供应链 …
  - `乔布斯` → 苹果 / 皮克斯 / 硅谷 …
- 这种结构带来极大的搜索不确定性，迫使模型在复杂网络中探索，而不是简单线性搜索。
- 最终可以得到一张较大的复杂知识图谱。

4️⃣ **随机游走采样子图**

- 在大图谱上进行随机游走，采样多个子图。
- 每个子图再生成一个 QA 对。

> **信息模糊化：在生成 QA 时，刻意引入模糊性，让模型必须通过 `推理 + 综合分析` 才能找到答案。**

1. 📅 **精确日期 → 模糊时段**（如 *2011年 → 2010年代初*）
2. 📝 **完整名称 → 部分掩码**（如 *Facebook → 名字以F开头的公司*）
3. 📊 **具体数值 → 定性描述**（如 *0.8% → 市场份额不足1%*）

这种形式迫使模型进行推理、比较、综合所有信息来得到最终答案，而不是简单的精确匹配。

> **基于此得到的数据集 SailorFog-QA，具有以下特点**

1. **基于真实互联网路径**：问题来源于真实搜索逻辑，符合人类的检索习惯。
2. **多样化推理模式**：复杂子图结构带来多步推理、组合分析、比较分析等多样推理方式。
3. **高度可扩展性**：随着图谱规模增长，可抽取的子图数量呈非线性增加，支持大规模数据合成。

> ✨ 总结：SailorFog-QA 的设计，让模型必须在复杂网络中不断推理，而不是靠简单记忆或匹配答案。

数据形式

首先规定一下数据集格式，依然采用广泛使用的 React 范式。具体来说，一条完整轨迹的格式如下

代码块

```
message = [
    {"role": "system", "content": system_message},
    {"role": "user", "content": user_prompt + question},

```

```
    {"role": "assistant", "content": '<think> 这里是思考过程 </think><tool_call> {"name": "工具名称", "arguments": {"参数名称": 参数值, "另一个参数名称": 另一个参数值, ...}} </tool_call>'},
    {"role": "user", "content": "<tool_response>\n" + result + "\n</tool_response>"},

    {"role": "assistant", "content": '<think> 这里是思考过程 </think><tool_call> {"name": "工具名称", "arguments": {"参数名称": 参数值, "另一个参数名称": 另一个参数值, ...}} </tool_call>'},
    {"role": "user", "content": "<tool_response>\n" + result + "\n</tool_response>"},

    ...

    {"role": "assistant", "content": '<think> 这里是思考过程 </think><answer> 答案 </answer>'},
]
```

- `system_message` 和 `user_prompt` 提示词的内容见下，`question` 为本次输入的问题

> 具体推理流程为：
>
> 1. 输入问题之后先进行一段思考，然后自己判断需要调用什么工具，将工具调用信息包围在 <tool_call>...</tool_call> 中
> 2. 调用工具，将返回结果放到 user 上作为新的输入，将工具返回结果包围在 <tool_response>...</tool_response> 中
> 3. 重复多次以上两个流程
> 4. 直到模型认为检索到的信息已经能够回答问题，那么就输出最终答案，包围在 <answer>...</answer> 中

- 工具包含两种：搜索(search)和访问(visit)
  - 搜索：调用谷歌搜索，输入参数为搜索 query，允许同时搜索多个 query，并为每个 query 返回前10个结果，每个结果包含一个标题、一段摘要和对应的 url。
  - 访问：访问特定网页，先获取该网页的完整内容，然后根据搜索的目标来进行针对性总结，这里由 Qwen2.5-72B 进行总结，同样允许访问多个网页，每个网页有自己的搜索目标。
- 最终完整轨迹的理论表示可以为如下形式，τ_i, a_i, o_i 分别表示第 i 步的思考过程、工具调用(行动 action)和工具返回结果(观测 observation)

$$\mathcal{H}_T = (\tau_0, a_0, o_0, \ldots, \tau_i, a_i, o_i, \ldots, \tau_T, a_T)$$

system_message

```
system_message = '''你是一位网络信息搜索大师。你的任务是彻底搜索互联网获取信息，并为问题提供准确的答案。无论查询多么复杂，你都不会放弃，直到找到相应的信息。

在进行过程中，请遵循以下原则：

1. **为答案持续行动**：你将进行多次互动，深入挖掘主题，探索所有可能的方面，直到找到令人满意的答案。

2. **重复验证**：在提供最终答案之前，你将**交叉检查**和**验证你收集的信息**，以确认其准确性和可靠性。

3. **关注细节**：你将仔细分析每个信息来源，确保所有数据都是最新的、相关的，并来自可信的来源。'''
```

user_prompt

```
user_prompt = '''用户和助手之间的对话。用户提出问题，助手通过调用以下一个或多个工具来解决问题。

<工具>
{
  "name": "search",
  "description": "执行批量网页搜索：提供一个查询数组；该工具在一次调用中为每个查询检索前10个结果。",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {
        "type": "array",
        "items": {
          "type": "string"
        },
        "description": "查询字符串数组。在一次调用中包含多个互补的搜索查询。"
      }
    },
    "required": [
      "query"
    ]
    }
},
{
  "name": "visit",
    "description": "访问网页并返回内容摘要。",
    "parameters": {
        "type": "object",
        "properties": {
```

```
            "url": {
                "type": "array",
                "items": {"type": "string"},
                "description": "要访问的网页的URL。可以是单个URL或URL数组。"
            },
            "goal": {
                "type": "string",
                "description": "访问网页的具体信息目标。"
            }
        },
        "required": [
            "url",
            "goal"
        ]
    }
}
</工具>

助手开始时进行一个或多个循环（思考使用哪个工具 -> 执行工具调用 -> 等待工具响应），最后以（思考答案 -> 回答问题）结束。思考过程、工具调用、工具响应和答案都包含在其标签内。可能有多个思考过程、工具调用、工具调用参数和工具响应参数。

示例响应:
<think> 这里是思考过程 </think>
<tool_call>
{"name": "工具名称", "arguments": {"参数名称": 参数值, "另一个参数名称": 另一个参数值, ...}}
</tool_call>
<tool_response>
工具响应
</tool_response>
<think> 这里是思考过程 </think>
<tool_call>
{"name": "另一个工具名称", "arguments": {...}}
</tool_call>
<tool_response>
工具响应
</tool_response>
（这里有更多思考过程、工具调用和工具响应）
<think> 这里是思考过程 </think>
<answer> 答案 </answer>

用户: '''
```

从 LRM 模型轨迹中重构推理

在得到复杂的 QA 对之后，下一步是获取中间的多步推理与工具调用过程，以便进行冷启动的 **SFT 训练**。

那么怎么得到中间的步骤呢？最直接的方法就是

1）使用现有推理模型（LRM）对问题进行采样，生成完整的推理轨迹。

2）通过拒绝采样保留正确的数据。

> 🌟 但是现在的推理模型存在以下两个问题
>
> 1. **风格污染：**现在 LRM 的输出具有明显的风格倾向，如果直接用来做微调，容易让模型学得“僵化”，降低泛化能力。
> 2. **上下文过长：**现在 LRM 的推理过程通常极其冗长，而且大家还追求更长（现有结论：推理越长正确率越高）。但本文的模型需要进行多步的工具调用，轮次和每轮的返回结果都很多，再加上过长的 CoT，很快会超出上下文限制。

因此，WebSailor 采用了一个折中办法：

- **先用强大的推理**模型：针对问题生成完整轨迹（中间可能有人工干预）。
- **再拒绝采样**：丢掉错误轨迹，保留正确轨迹。
- **去除冗余思考过**程：只保留 工具*调用方式* 和 *返回结果*。
- **用 Instruct 模型重构推理**：
  - 输入 = 历史信息 + 下一步工具调用及返回结果。
  - 输出 = 简洁、合理的“思考过程”。

$$\hat{\tau}_t \sim \pi^*(\tau|\mathcal{H}_{t-1}, a_t, o_t).$$

冷启动强化学习训练

训练分为两个阶段：

1. **SFT 冷启动**：让模型具备基本的工具使用能力，学会多轮推理模式。
2. **强化学习 (RL)**：在此基础上进一步提升推理稳定性与正确率。

拒绝采样微调

首先使用三种方式的拒绝采样来保证冷启动数据集的优质性

1. ✅ 只保留最终得到正确答案的轨迹。

2. ✂ 丢弃任何长度超过 **32k token** 的轨迹（因为蒸馏的教师模型更强，有更长的上下文，但无法训练学生模型）
3. 🔄只保留工具调用超过 5 次的轨迹，以保证任务的复杂性

⚠ **特别注意：**这里单独提了一点，但也是大家的共识了，就是在训练的时候要把 <tool_response>工具响应</tool_response>的部分给 mask 掉。

强化学习算法： DUPO（Duplicating Sampling Policy Optimization）

在强化学习阶段，模型需要为每个输入生成多个轨迹（即 **rollout**），再根据轨迹优劣来更新策略。但这里有个难题：

- 如果某个输入的轨迹 **全对** 或 **全错**，对模型训练毫无帮助，因为无法提供优化方向。

> 🎁 **DAPO 提出**
>
> - 剔除“全对/全错”的样本，再用新的样本补齐批次大小（不补充到批次大小会导致训练不稳定），这样数据的利用率更高，最终的效果也更好。
> - 但问题是：WebSailor 的轨迹往往很长（数十次工具调用），补充数据重新 rollout 会导致训练速度大幅下降。

DUPO 新方案 🚀

- 同样剔除“全对/全错”的样本，但不再补充新数据，而是**从同一批次的有效样本中随机抽取进行复制**。这样最多只是那些样本学了两次，并不会降低很多性能，但是这种方式使训练速度提升了 2~3倍。

公式和 DAPO 是一样的，只是动态采样这里做了优化：

$$\mathcal{J}(\theta) = \mathbb{E}_{(q,y)\sim\mathcal{D},\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|\text{context})}$$
$$\left[\frac{1}{\sum_{i=1}^{G}|o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\min\left(r_{i,t}(\theta)\hat{A}_{i,t}, \text{clip}\left(r_{i,t}(\theta), 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}}\right)\hat{A}_{i,t}\right)\right]$$
$$r_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t} \mid \text{context})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid \text{context})}, \quad \hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^{G})}{\text{std}(\{R_i\}_{i=1}^{G})}.$$

这里 o_i 表示模型生成的 token，而 $context$ 还包括了历史的工具返回结果。

为了避免 **reward hacking**，WebSailor 没有使用奖励模型，而是只采用了规则式奖励，包括格式奖励和正确性奖励：

$$R_i = 0.1 * R_i^{format} + 0.9 * R_i^{answer}.$$

- **格式奖励**：检查轨迹是否遵循格式规范（如 `<think>` 标签是否正确包裹）。
- **正确性奖励**：用 LLM 判断预测答案是否与真实答案一致。

实验

实验设置

Benchmarks

- **BrowseComp-en**：OpenAI 推出的超高难度测试集。
- **BrowseComp-zh**：BrowseComp-en 的中文版。
- **GAIA**：一个需要多模态与工具使用的测试集，本文仅使用其中的文本部分，共 103 条数据。
- **XbenchDeepSearch**：专注评估智能体的工具使用能力，尤其是深度信息检索和复杂搜索任务。

模型：Qwen-2.5-3B、Qwen-2.5-7B、Qwen-2.5-32B、Qwen-2.5-72B

Baselines

- **直接推理：**用模型内部知识回答，**非推理模型**包括 Qwen-2.5-32B、Qwen-2.5-72B、GPT-4o、GPT-4.1；
  推理模型包括 QWQ-32B、o4-mini、DeepSeek-R1。不考虑较小的模型，因为在 BrowseComp 上的得分基本为零。
- **专有浏览 agent：**OpenAI DeepResearch、Grok-DeepResearch、带思考和搜索的Doubao。
- **开源 agent：**Search-o1、WebThinker、R1-Searcher、WebDancer。

超参数：temperature=0.6，top-p=0.95

评价指标：使用 pass@1 和 pass@k 作为评价指标

$$\text{pass@1} = \frac{1}{n}\sum_{i=1}^{n} p_i$$

其中：

- n = 数据集大小
- p_i = 第 i 个样本一次生成结果是否正确
- pass@k：同一样本生成 k 次，有一次正确即算回答正确。

实验结果

1. **直接使用内部知识** → 表现极差 ❌
   - 因为问题普遍需要外部检索，模型“死记硬背”完全不够。
2. **推理模型** → 有一定潜力 ✨
   - 部分问题模型内部其实知道答案，通过长 CoT 推理，有时能得到正确结果。
3. **WebSailor 表现** → 显著提升 🚀

- 尤其是 **WebSailor-72B**，相比其他模型分数大幅提高，说明这种“高难数据集 + 强化学习”的训练范式是有效的。

4. **在 GAIA 上的结果** → 提升有限 📉
   - GAIA 中含有大量数学与计算任务，WebSailor 并未针对这类任务优化，因此表现一般。

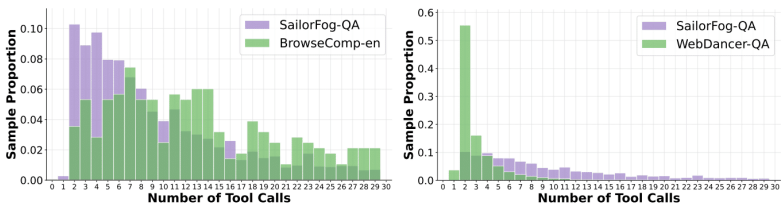
> ✨ 小结：虽然与 **DeepResearch** 尚有差距，但 WebSailor 已经展示出开源模型在复杂网页推理上的巨大潜力。

Table 1: Main results on four challenging benchmarks. ‡ indicates that these proprietary methods are manually evaluated through their websites (some are reported in the corresponding benchmark papers). - means that we do not have the results due to cost constraints.

Backbone	Paradigm	BrowseComp-en	BrowseComp-zh	Xbench-DeepSearch	GAIA
Direct Inference					
Qwen-2.5-32B	Direct	0.6	3.9	8.7	13.6
Qwen-2.5-72B	Direct	0.6	7.0	12.7	14.6
GPT-4o	Direct	0.6	6.2	18.0	17.5
GPT-4.1	Direct	1.5	14.4	17.0	22.3
QwQ-32B	Direct	0.5	10.0	10.7	22.3
o4-mini	Direct	**6.1**	15.2	22.3	**33.3**
DeepSeek-R1	Direct	2.0	**26.3**	**32.7**	16.5
Proprietary Agents					
Grok-3‡	Browsing	-	12.9	50+	-
Doubao‡	Browsing	-	26.0	50+	-
GPT-4o‡	Browsing	1.9	-	-	-
DeepResearch‡	Browsing	**51.5**	**42.9**	-	**67.4**
Open-source Agents					
R1-Searcher-7B	ReAct	0.4	0.6	4.0	20.4
Qwen-2.5-32B	Search-o1	0.1	2.4	3.7	28.2
WebDancer-32B	ReAct	2.5	14.1	38.7	40.7
QwQ-32B	Search-o1	2.8	17.9	25.0	39.8
WebThinker-RL	ReAct	2.8	7.3	24.0	48.5
WebDancer-QwQ	ReAct	3.8	18.0	39.0	51.5
WebSailor-3B	ReAct	3.3	9.7	27.7	33.0
WebSailor-7B	ReAct	6.7	14.2	34.3	37.9
WebSailor-32B	ReAct	10.5	25.5	53.3	53.2
WebSailor-72B	ReAct	**12.0**	**30.1**	**55.0**	**55.4**

消融研究

SailorFog-QA 的复杂性： 图中展示了不同数据集中工具调用次数的对比，可以看出 SailorFog-QA 的分布与 BrowseComp-en 更相似，难度远远超过 WebDancer-QA（qwen前一个浏览agent的工作），说明 SailorFog-QA 的复杂性更高。需要注意的是，模型在训练的时候只使用工具调用次数超过 5 次的轨迹。



(a) Comparison with BrowseComp-en (b) Comparison with WebDancer

Figure 3: A comparison of the number of tool calls in our training set with those in the training sets of WebDancer and BrowseComp-en.

SailorFog-QA 的通过率： 下表展示在配备浏览工具和 ReAct 框架下，o4-mini 和 DeepSeek-R1 在不同数据集的表现，可以看出 WebDancer-QA 的数据集还是太简单了，直接推理的分数就很高。相比之

下 SailorFog-QA 就更难一些，如果将难度进一步加大到 BrowseComp-en 的水平，说不定表现会更好。

Table 2: The pass@1 accuracy of the SailorFog-QA, the WebDancer training set, and BrowseComp-en under the ReAct framework.

Backbone	SailorFog-QA	WebDancer-QA	BrowseComp-en
o4-mini	47.3	90.2	26.3
DeepSeek-R1	38.9	84.4	9.5

简单任务的兼容性：在前文对不同任务的难度进行了定义，本文模型主要增强三级任务的能力，上面评测的 Benchmarks 也基本都是二级或三级任务。为了验证这种方法训练的模型在一级任务上能力有没有退化，在 SimpleQA 数据集上进行了测试，可以看出 WebSailor 表现最好，说明这种训练方面具有向后兼容性。

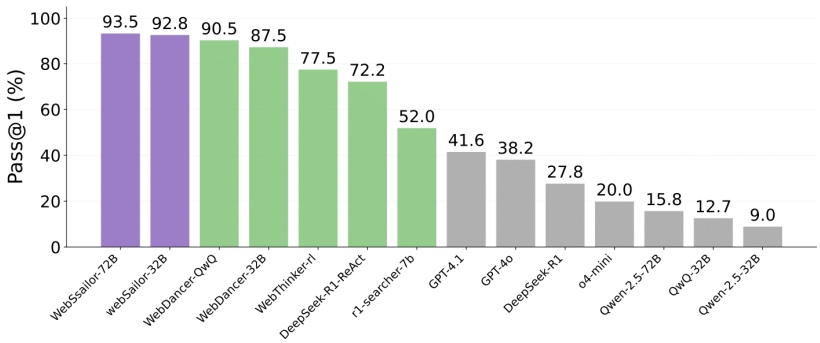


Figure 4: Performance on the SimpleQA benchmark.

强化学习的提升：下图比较强化学习前后 pass@1 和 pass@3 的变化，首先可以看出在所有情况 RL 都带来了提升。RL 对 pass@1 的提升远高于 pass@3，说明 RL 对正确轨迹的稳定生成进行了强化。

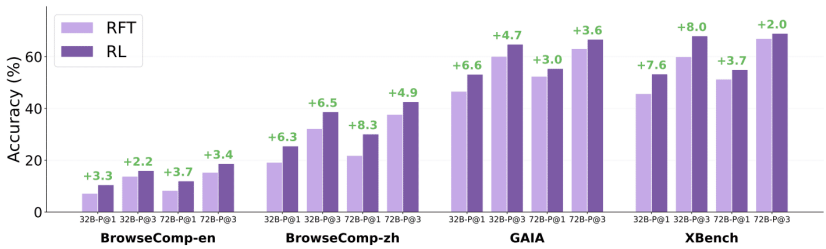
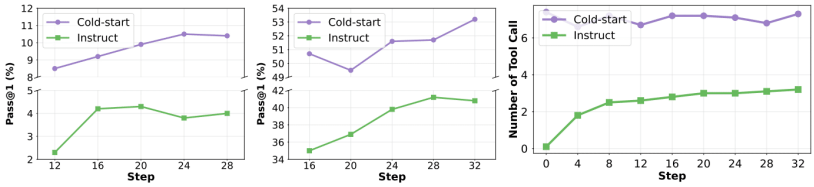


Figure 5: Detailed evaluation results using Pass@1, Pass@3.

冷启动的必要性：下图展示 RFT 冷启动 +RL 和直接 RL 的对比，可以看出冷启动的模型在最终性能上会更好。虽然直接强化学习也会有很大的提升，但是增长很有限，包括工具调用次数上，最高只能维持在不到4次上。



(a) Pass@1 on BrowseComp-en. (b) Pass@1 on GAIA. (c) Change in the number of tool calls.

Figure 6: Comparing direct RL training of Qwen-2.5-instruct-32B with RL training after an RFT cold start.

局限性和未来工作

1. **上下文长度限制** 📏
   - WebSailor 的最大上下文为 32K，对于一些长任务仍不够，很多错误都源于超长上下文。
   - 随着推理长度增加，性能可能下降，这也反映出当前大模型在长文本处理上的不足。
2. **过度思考的倾向** 🤯
   - 对于简单问题，WebSailor 也会进行多步工具调用。
   - 但这未必是坏事，因为它往往是在进行交叉验证，利用多信息源来确认结果。
3. **强化学习效率低** 🐢
   - WebSailor 的 RL 仅训练了 50 个 step，这主要是因为同步 RL 框架效率太低，即使用 DUPO 也依旧缓慢。
   - 未来重点是开发 **异步训练框架**，以便在更大规模上训练。

总结

- WebSailor 验证了 **通过构造超难数据集+强化学习** 可以解决 DeepResearch 推理任务，这就给其他人提供了思路。
  - 我们也可以尝试构造一个更难质量更高的数据集，来进一步提高模型在这个领域的性能。
- 在线训练的强化学习随着 DeepSeek-R1 的发布又引起了广泛关注（相比于此前更受欢迎的 DPO）。
- 强化学习框架（verl、OpenRLHF ）的训练效率还是有点低，未来也还有非常多的优化空间。

> ✨ Takeaway：开源模型在网页推理上已有突破，但若要追赶 DeepResearch，关键在于 **更强的数据集** + **更高效的 RL 框架**。